



守护线程

北京理工大学计算机学院
金旭亮

线程决定进程的“生命周期”

默认情况下，如果一个进程启动了多个线程（这些线程称为“前台线程”），那么，只要有一个线程还在运行中，进程就还“活着”。

```
//线程函数
2 usages
static Runnable runnable = () -> {
    while (true) {
        System.out.println("线程正在运行中...");
        try {
            Thread.sleep( millis: 1000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
};
```

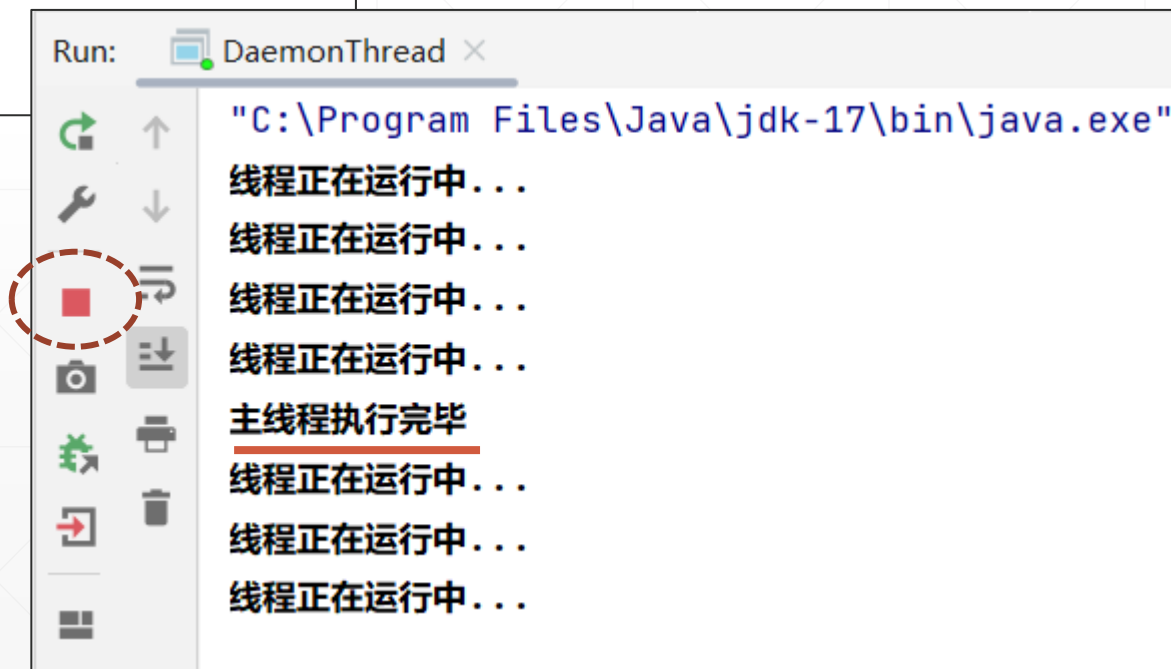
```
private static void useForegroundThread()
    throws InterruptedException {
    new Thread(runnable).start();
    Thread.sleep( millis: 4000);
    System.out.println("主线程执行完毕");
}
```

主线程调用useForegroundThread方法，等4秒后退出.....

运行截图

```
public static void main(String[] args)
    throws InterruptedException {
    //当进程结束时, 输出相应信息
    Runtime.getRuntime().addShutdownHook(
        new Thread(() -> System.out.println("进程已经结束")));
    useForegroundThread();
}
```

虽然主线程已经结束, 但启动的线程
由于里面有一个while(true)循环,
会导致整个进程迟迟无法结束.....



Daemon线程（守护线程）



Daemon线程是在后台执行的“背景”线程，当主线程结束时JVM会自动结束这个线程。



默认情况下，创建的线程均为“前台”线程，需要调用 `setDaemon` 方法设置为“背景”线程：

```
thread.setDaemon(true);
```



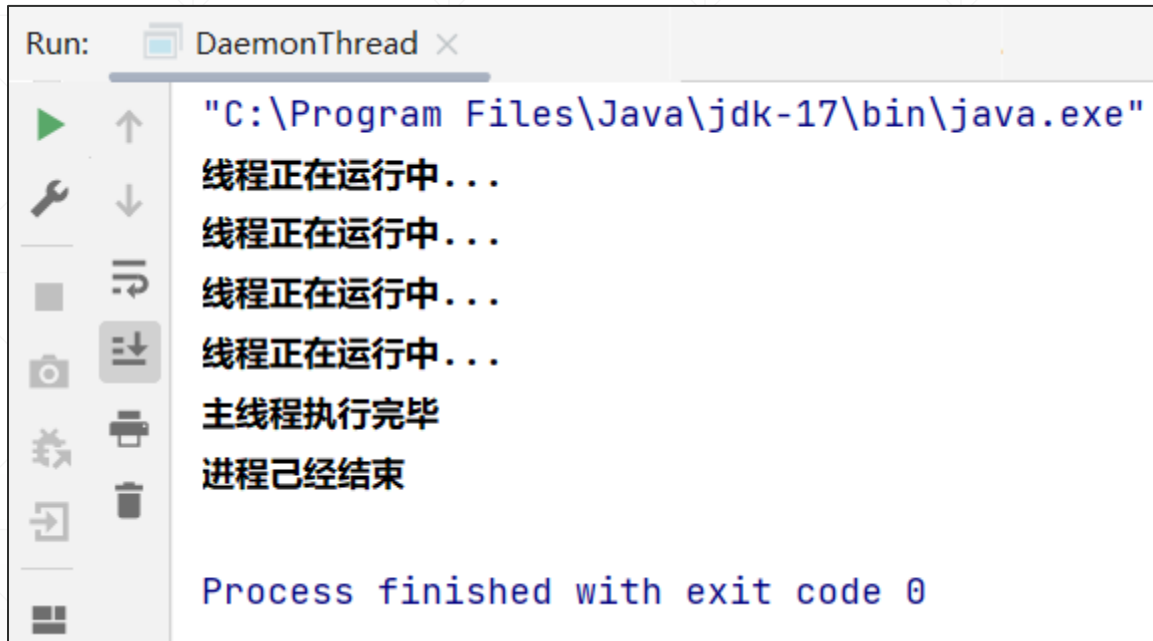
通过 `isDaemon()` 方法可以检查线程是一个守护线程（此时方法返回 `true`）还是一个非守护线程（此时方法返回为 `false`）。



`setDaemon()` 方法只能在 `start()` 方法之前调用，一旦线程开始执行，其 `daemon` 状态便不可修改。此时调用 `setDaemon()` 方法，将抛出 `IllegalThreadStateException` 异常。

使用守护线程

```
1 usage
private static void useDeamonThread()
    throws InterruptedException {
    var thread = new Thread(runnable);
    thread.setDaemon(true);
    thread.start();
    Thread.sleep( millis: 4000 );
    System.out.println("主线程执行完毕");
}
```



从程序截图可以看到，如果主线程启动的线程是守护线程，则当主线程一退出，守护线程也随之结束.....

守护线程的应用场景



在JVM中垃圾回收器就采用了守护线程



在一些中间件的心跳检测、事件监听等涉及定时异步执行的场景中也可以使用守护线程



对于一些后台任务，当不希望阻止JVM进程结束时，可以采用守护线程。